Zaguik: Zero-Knowledge Attestation for Privacy-Preserving LLM Governance

Jan Aguiló Plana
Universitat Pompeu Fabra
Carrer de Roc Boronat, 138, Barcelona
jan.aguilo01@estudiant.upf.edu

Vanesa Daza
Universitat Pompeu Fabra
Carrer de Roc Boronat, 138, Barcelona
vanesa.daza@upf.edu

***Abstract*—Regulators increasingly require evidence-backed assurances about how AI systems handle sensitive information. We present Zaguik, a privacy-preserving attestation pipeline that proves, in zero knowledge, that a large language model (LLM) response passed a personally identifiable information (PII) filter, without revealing either the response text or the filtering model. The system generates an LLM response, applies a PII detection policy, and produces a zk-SNARK using EzKL over the detector inference. The proof exposes only a public commitment (a hash) to the exact response, binding verification to the delivered output while keeping its contents private; filtering logic and outputs remain private. We provide a runnable reference implementation, discuss practical setup requirements and ONNX compatibility constraints, and outline how such proofs can support GDPR-aligned auditing and transparent AI governance.**

***Index Terms*—Jornadas, Ciberseguridad**

Tipo de contribución: *Investigación en desarrollo*

I. Introduction

Artificial intelligence (AI) systems that process personal data are increasingly subject to enforceable obligations beyond voluntary transparency pledges. Regulations such as the European Union's General Data Protection Regulation (GDPR) and emerging AI acts require evidence-backed assurances about how systems handle sensitive information, comply with safety requirements, and respect data-subject rights. For large language models (LLMs) deployed in customer-facing or regulated settings, demonstrating that outputs do not leak personally identifiable information (PII) is both a legal necessity and a trust requirement. Auditors and regulators need not only policies on paper but verifiable guarantees that those policies are actually applied in production.

Today, verifying that a provider applies a PII filter to LLM outputs typically requires the auditor to trust the provider's claims or to inspect the outputs and the filtering logic directly. If the auditor sees the raw outputs, privacy is compromised; if the provider must disclose the filtering model or its parameters, intellectual property and competitive concerns arise. This tension leaves a gap: there is no cryptographically sound way to prove that a given output was filtered for PII without revealing either the output or the internals of the filter. Zero-knowledge proofs (ZKPs) address this need by allowing one party (the prover) to convince another (the verifier) that a statement is true without revealing the underlying data or computation. Applied to AI pipelines, ZKPs can attest that a specific computation was performed (e.g., that a PII filter was applied and that its result indicated no PII) while keeping the model and the actual text private.

In this work we describe and implement Zaguik, a system that produces such a verifiable, privacy-preserving attestation for LLM outputs. The pipeline works as follows: (1) an LLM generates a response to a user prompt; (2) a PII filter (in our reference implementation, a regex-based detector) is applied to the output; (3) if the output passes the filter, a SHA256 hash of the output is computed and the output is encoded as input to a circuit; (4) a zk-SNARK proof is generated using EzKL, attesting that the filter was applied and that the circuit output corresponds to a "clean" result, with the proof bound to the hash of the response; (5) any verifier holding the hash and the proof can confirm that filtering was correctly applied without ever seeing the text or the filter internals. The proof thus ties verification to the exact delivered output while preserving confidentiality. This approach illustrates how cryptographic attestation can support transparent AI governance: compliance and audit can be based on verifiable proofs rather than on trust alone, and both user privacy and provider confidentiality can be preserved. We provide a runnable reference implementation and discuss practical considerations.

II. Related Work

A. Zero-Knowledge Proof Systems

Zero-knowledge proof systems such as Groth16 [1], PLONK [2]; and related zk-SNARK constructions provide succinct and efficiently verifiable proofs of correct computation over arithmetic circuits. These systems allow a prover to convince a verifier that a computation was executed correctly without revealing private inputs.

More recently, practical toolchains such as libsnark [3], Halo2 (Bowe et al., 2020), and EzKL (2023) have enabled developers to compile high-level programs and machine learning models into circuits suitable for zk-SNARK proving.

However, these works focus on the cryptographic machinery for proving circuit execution correctness. They do not address the governance question of how such proofs can be used to attest enforcement of content filtering policies over LLM outputs without revealing those outputs.

B. Zero-Knowledge Machine Learning (ZKML)

Zero-knowledge machine learning focuses on proving correct execution of machine learning inference without revealing private inputs. zkCNN [4] demonstrates how convolutional neural network predictions and accuracy can be verified in zero knowledge by expressing inference as arithmetic circuits.

Similarly, vCNN [5] presents a framework for generating zk-SNARK proofs that certify correct execution of convolutional neural network inference.

These works address inference correctness: proving that a model's output was computed faithfully according to fixed parameters. In contrast, our work does not aim to prove correct execution of the LLM itself. Instead, we focus on attesting enforcement of a post-generation filtering policy over LLM outputs. To our knowledge, prior ZKML research does not address governance-level attestation of output filtering for generative models, where the goal is not to prove the model computation but to prove that a compliance constraint was applied to the generated output.

C. Privacy-Preserving Machine Learning and Inference

Research in privacy-preserving machine learning includes techniques such as secure multiparty computation [6], homomorphic encryption for neural network inference [7], [8], and confidential computing enclaves.

These approaches focus on protecting either model parameters or user inputs during inference. For example, homomorphic encryption allows inference over encrypted data without revealing the input, while secure multiparty computation distributes trust across multiple parties.

Our work differs in focus: rather than protecting model inputs or parameters, we aim to protect the model output while still enabling verifiable policy enforcement. The output remains private, yet a verifier can be assured that a specified constraint was checked.

D. AI Auditing, Transparency, and Governance

AI governance frameworks emphasize auditing, transparency, and accountability. Model cards [9], datasheets for datasets [10], and regulatory frameworks such as the EU AI Act propose documentation-based or process-based accountability.

In practice, auditing of deployed systems typically requires access to logs, outputs, or internal documentation. Such approaches rely on institutional trust and do not provide cryptographic guarantees of enforcement at runtime.

Recent discussions in AI safety research have proposed verifiable AI or cryptographic attestation mechanisms, but concrete implementations for generative output filtering remain limited.

Our work contributes a concrete mechanism for cryptographically attesting that a filtering policy was executed on a specific LLM output, without revealing that output to the verifier.

E. PII Detection and Content Moderation

Personal data detection is commonly performed using rule-based systems (regex patterns), named entity recognition models, or hybrid approaches [11]. Production systems for content moderation rely on deterministic filters, ML classifiers, and policy enforcement pipelines.

However, these systems provide enforcement but not verifiability. An external auditor must either trust the system operator or inspect outputs directly. To our knowledge, prior work on PII filtering does not provide a mechanism for cryptographically proving that filtering was applied to a given output without revealing the output itself.

F. Positioning Statement

To our knowledge, no prior work combines zero-knowledge proofs with LLM output filtering to provide privacy-preserving attestation of policy enforcement. Existing zk-SNARK and ZKML systems enable proving correct circuit execution, but they do not address governance-level questions of output compliance. Conversely, AI auditing and moderation systems enforce policies but lack cryptographic verifiability.

Our work occupies the intersection of these domains: we demonstrate a practical prototype that attests enforcement of an output constraint over LLM-generated text while preserving output confidentiality.

III. Methodology

The core contribution of this work is the *cryptographic attestation pipeline*: we show how to produce a zk-SNARK proof that a given string was passed through a certified check, with the proof bound to a commitment to the output and with no disclosure of the text or the checker's internals. The PII filter is the policy we attest to; its design is deliberately simple so that the focus remains on the proof system. Below we describe the architecture, the filter used in the current implementation, the encoding of outputs for the circuit, the proof system and visibility semantics, and the setup, proof, and verification procedures.

A. System Overview

The pipeline has two phases: a *filtering phase* and a *proof phase*. In the filtering phase, a user prompt is answered by an LLM (in our implementation we use DistilGPT-2); the response is then checked by a PII detector. Only if the detector reports that the text contains no PII do we enter the proof phase. There we encode the (clean) output as input to a circuit that models the same filtering logic, generate a zk-SNARK proof over that circuit using EzKL, and output a proof bound to a public commitment (hash) of the response. A verifier who receives the proof and the commitment can check that the filtering computation was correctly applied to the committed output, without learning the text or the internal parameters of the filter. The LLM itself is out of scope of the proof: we attest only that a given string was passed through the certified PII filter and passed.

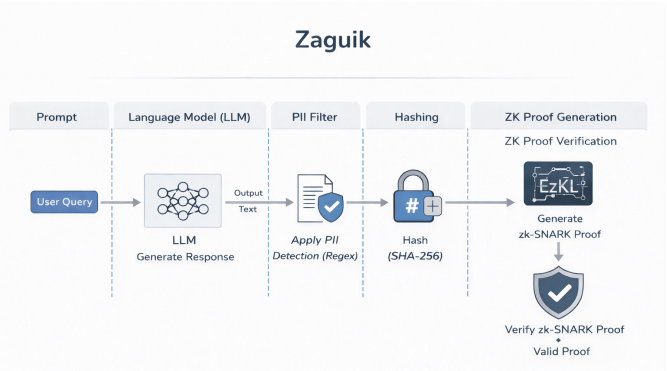

Figure 1. End-to-end workflow: from user prompt and LLM response through PII filtering to proof generation and verification.

B. Formal Statement and Security Model

We formally define the statement that Zaguik proves and clarify the visibility of data in the zero-knowledge setting.

a) Notation.: Let $y \in \{0,1\}^*$ denote the output string generated by the LLM. Let $H(\cdot)$ be a cryptographic hash function (SHA256 in our implementation). Let F denote the PII filtering policy. Let C be the certified arithmetic circuit that implements F. Let L be the fixed input length in bytes used for encoding (in our implementation $L = 2048$). Let $\mathrm{Enc}_L(y)$ denote the byte-level encoding of y, truncated or zero-padded to length L.

b) Public and Private Data.: The zero-knowledge proof distinguishes between public inputs and private witness:

- **Public input:**
  - $h = H(y)$, a commitment to the LLM output.
  - The verification key associated with circuit C.
- **Private witness:**
  - The full LLM output string y.
  - Its encoded representation $x = \mathrm{Enc}_L(y)$.
  - The internal witness values required to execute circuit C.

The verifier learns only the commitment h and the proof. The underlying text y and the internal circuit execution remain hidden.

c) Language Definition.: The proof system attests membership in the following NP language:

$$\mathcal{L} = \{\, h \mid \exists y \text{ such that } H(y) = h \wedge C(\mathrm{Enc}_L(y)) = 1 \,\}.$$

In other words, $\mathcal{L}$ contains all hash values for which there exists a corresponding text that both matches the hash and passes the certified PII filtering circuit.

d) Zero-Knowledge Statement.: The prover demonstrates, in zero knowledge, that:

$$\exists y \text{ such that } H(y) = h \wedge C(\mathrm{Enc}_L(y)) = 1.$$

Informally, this means:

> "There exists an LLM output y that corresponds to the public hash h, and when processed by the certified PII filtering circuit C, the result is `PASS`."

In our current implementation, the consistency between the public hash h and the encoded input $\mathrm{Enc}_L(y)$ is enforced at the protocol level: the prover computes both values from the same output string prior to proof generation. The hash equality is not verified inside the arithmetic circuit.

e) Scope of the Guarantee.: The proof guarantees correct execution of the certified filtering circuit on the committed output. It does not guarantee that the filtering policy is complete or semantically sufficient, nor does it make claims about the internal behavior of the LLM. The guarantee is limited to enforcement integrity: that the specific circuit C was correctly executed on a text consistent with the public commitment.

C. PII Filtering and Policy

We distinguish clearly between two components of the filtering pipeline: an operational PII detector used in plain execution, and a circuit-friendly policy used for zero-knowledge attestation.

First, the system applies a regex-based PII filter in plain execution. This detector searches for structured patterns such as email addresses, phone numbers, identification numbers, and other commonly formatted personal data. Its role is operational: it acts as a gating mechanism, preventing obviously unsafe outputs from proceeding to the proof generation stage.

Second, for zero-knowledge attestation, we implement a separate, circuit-friendly policy as a small PyTorch module that is exported to ONNX and compiled into an arithmetic circuit using EzKL. This module operates at the byte level. It takes as input a fixed-length normalized byte vector and counts occurrences of selected "suspicious" byte values (e.g., '@', '-', '.', and digits). The circuit outputs a scalar count, and the policy is considered satisfied if this count is zero.

The zk-SNARK proof certifies correct execution of this ONNX-derived byte-level circuit and the fact that its output equals zero. It does not certify direct execution of the plain regex filter. The regex detector and the byte-counter circuit are separate mechanisms: the former provides practical PII detection in deployment, while the latter encodes a structurally checkable constraint that can be efficiently represented and verified within an arithmetic circuit.

It is therefore important to clarify the scope of equivalence. The proof guarantees only that the ONNX circuit was executed correctly on the encoded output. The correspondence between the plain regex-based filter and the byte-level circuit is a design choice, not a formally verified equivalence. The byte-counter policy can be interpreted as a conservative structural proxy for certain classes of PII indicators, but it does not capture the full semantic expressiveness of the regex detector. Establishing formal semantic equivalence between high-level filtering logic and its circuit representation remains an open research problem and is outside the scope of this prototype.

The attestation mechanism itself is policy-agnostic: any deterministic filtering rule that can be expressed as an ONNX model compatible with the proof backend can be certified in the same manner. In this work, the simplified byte-level policy serves to demonstrate the feasibility of zero-knowledge attestation for LLM output governance under current practical constraints.

D. Encoding and Circuit Input

The LLM output is a string. To feed it into the circuit we encode it in fixed-size form. We fix an upper bound L on the number of UTF-8 bytes (in our implementation L = 2048). The LLM generation process is configured so that attested outputs do not exceed L bytes. Therefore, for every attested output, the circuit receives the entire string.

We use a byte-level encoding: the UTF-8 representation of the string is zero-padded to length L if shorter. Each byte is normalized to [0,1] by dividing by 255. The resulting vector has shape [1, L] and is used as the single input tensor to the ONNX circuit.

The same string that passes the plain PII check is encoded and provided as private witness input to the circuit. The circuit operates over this full encoded representation.

Separately, we compute a SHA256 hash of the complete output string. This hash serves as a public commitment that binds the proof to the exact delivered response. The hash is not used as circuit input; instead, it is provided as public data

alongside the proof so that external parties can associate the attestation with a specific output.

E. Proof System and Visibility Semantics

We use EzKL to compile the ONNX model into an arithmetic circuit and to produce zk-SNARK proofs (succinct, non-interactive arguments of knowledge). The security and privacy guarantees depend on how inputs, outputs, and parameters are exposed.

The encoded *input* is kept private as witness data. The verifier learns only the zk-SNARK proof and the public hash commitment: the prover commits to the encoded output string, and the verifier learns this commitment but not the underlying bytes, so the LLM response stays private while the proof is bound to a specific output. The *output* of the circuit is made available to the verifier so they can check that it indicates "pass" (e.g., zero PII score). The filter *parameters* are fixed in the circuit and not revealed; the verifier only learns that the same certified circuit was executed. Together, these choices ensure that we prove "this committed output was run through the certified PII filter and passed" without leaking the text or the filter internals.

F. Setup and Proof Generation

Setup is performed once per circuit (e.g., per ONNX model and settings). The ONNX is used to generate the proof-system settings and to compile an arithmetic circuit; a KZG structured reference string (SRS) is required and can be obtained from a trusted source or generated locally for testing. With the compiled circuit and SRS, key generation yields a proving key and a verification key.

For each clean LLM output we wish to attest, we encode it as input data, compute the witness for the circuit, and run the prover to produce the final proof. The proof, the verification key, and (in deployment) the public commitment to the output are what the verifier needs.

G. Verification

Verification is deterministic: given the proof, the verification key, the settings, and the SRS, the verifier runs the verification algorithm and accepts if and only if the proof is valid. No secret data is required. If the verifier holds a hash of the LLM output (e.g., SHA256 or the in-circuit commitment), they can confirm that the proof applies to that exact output, so that attestation is non-detachable and binding.

H. Implementation and Reproducibility

Our reference implementation is in Python. The PII filter is regex-based in the current experiments. The cryptographic pipeline (regex-derived ONNX export, encoding, setup, proof generation, and verification) is implemented as a single scriptable workflow. The default configuration builds the small regex-derived ONNX, runs setup, and then generates and verifies proofs for a given text string. All steps are documented so that the pipeline can be reproduced and extended; integrating a more expressive PII model would require an ONNX export that is compatible with the proof system backend.

Table I
ONE-TIME SETUP COSTS (REGEX PII ONNX).

Step	Time (s)
Settings generation	0.07
Circuit compilation	0.03
SRS retrieval	0.00
Witness (setup)	0.07
Key generation	12.28
Total	12.45

Table II
PROOF SIZE AND TIMINGS VS. LLM OUTPUT LENGTH (MAX TOKENS).

Max tokens	Proof size (KB)	Proof gen. (s)	Witness (ms)	Verify (ms)
30	21.4	9.85	41	180
50	21.4	9.83	42	149
100	21.5	8.56	40	151
200	21.4	8.58	41	142
300	21.4	9.72	39	170
500	21.4	13.08	53	157

IV. RESULTS

We ran the full pipeline (DistilGPT-2, regex PII filter, proof generation and verification) with a fixed prompt and varied the maximum number of tokens generated by the LLM to assess how proof size and timings behave with respect to output length. All runs used the same regex-derived ONNX circuit and the same one-time setup. Setup was performed once; the reported setup costs are from a single run and include generating settings, compiling the circuit, obtaining the SRS, and key generation. Table I summarizes these one-time costs. Table II reports, for each output length (max tokens), the proof size, total proof generation time, witness generation time, and verification time.

Proof size remains stable at 21.4 and 21.5 KB across all output lengths, as expected for a fixed circuit: the zk-SNARK proof size depends on the circuit structure, not on the input length. Proof generation time is dominant (roughly 8–13 s in these runs); witness generation stays under 55 ms and verification under 180 ms. The reported circuit corresponds to a lightweight regex-based filter compiled to ONNX; more complex neural PII detectors would increase circuit size and proof cost. These results indicate that zero-knowledge attestation of LLM output filtering is computationally feasible for small fixed circuits, with proof sizes around 21 KB and verification times below 200 ms, making third-party verification lightweight relative to proof generation.

V. DISCUSSION

A. Interpretation of Results

The experimental results demonstrate that zero-knowledge attestation of LLM output filtering is computationally feasible using a small, fixed circuit. Proof size remains constant (approximately 21 KB) across varying output lengths, confirming that zk-SNARK size depends on circuit structure rather than input data. This aligns with the theoretical properties of KZG-based proving systems such as Halo2.

Proof generation time is the dominant cost (approximately 8–13 seconds in our experiments), while verification remains lightweight (below 200 ms). This asymmetry is desirable in

governance settings: proof generation occurs on the provider side, whereas verification can be performed by auditors, regulators, or third parties with minimal computational overhead. Witness generation time remains negligible compared to proving, further indicating that the cryptographic proving step is the primary computational bottleneck.

Because the circuit operates on a fixed-size input encoding ($L = 2048$ bytes), LLM outputs longer than L are truncated by the pipeline before attestation; outputs shorter than L are zero-padded. The bound L is chosen so that the *whole* (capped) output is encoded and attested in a single proof, avoiding chunking. As a result, the constraint system remains constant across experiments, explaining the stability of proof size and runtime.

B. Governance and Security Implications

The proposed pipeline illustrates a shift from trust-based enforcement to cryptographically verifiable enforcement. In conventional deployments, external parties must trust that providers correctly apply safety or privacy filters to LLM outputs. Alternatively, verification requires exposing either the generated output or the filtering logic, creating privacy and intellectual property risks.

By contrast, our approach enables a third party to verify that a filtering policy was applied correctly to a specific output (bound via a cryptographic hash) without revealing the output itself or the internal logic of the filter. This supports a form of privacy-preserving auditability aligned with regulatory requirements such as GDPR, particularly in contexts where outputs may contain sensitive information.

The mechanism is policy-agnostic: although we instantiate it with a PII regex filter, the same architecture can, in principle, attest to toxicity detection, policy compliance, or other post-processing safeguards.

C. Limitations

Several limitations must be acknowledged.

First, the filtering policy implemented in this prototype is lightweight and regex-based. While sufficient to demonstrate feasibility, it does not reflect the complexity of modern neural PII detection systems. Attempts to integrate larger ONNX-based NER models revealed compatibility constraints within current ZKML tooling. More expressive models would likely increase circuit size and proof cost.

Second, the circuit operates on fixed-length inputs. Outputs are capped at L so that a single proof attests to the entire string; longer responses would require chunking, multiple proofs, or recursive composition to remain practical.

Third, the system proves correct execution of the filtering policy, not the adequacy of the policy itself. A weak or incomplete PII detector can still be correctly executed and proven. Thus, zero-knowledge attestation ensures enforcement integrity, but not semantic completeness.

These limitations define important boundaries for interpreting the results and guide future extensions.

VI. Conclusions

We presented Zaguik, a privacy-preserving attestation pipeline that proves, in zero knowledge, that an LLM output has passed a PII filtering policy without revealing either the output or the filtering logic. Using EzKL and a fixed ONNX-based circuit, we demonstrated that proof sizes remain constant and verification costs remain low, while proof generation is computationally manageable for backend deployment.

These results indicate that zero-knowledge proofs can serve as a practical mechanism for verifiable AI governance, enabling auditability without exposure. Beyond standalone LLM responses, such attestation mechanisms could be integrated into agentic workflows, where language models trigger actions, access tools, or store intermediate state. In these settings, cryptographic guarantees that each intermediate output satisfies policy constraints could provide a foundational layer for privacy-preserving and accountable AI systems.

Future work will explore integration with neural PII detectors, scalable handling of longer outputs, and deployment within broader AI compliance frameworks.

References

[1] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology – EUROCRYPT 2016*, vol. 9666, 2016.

[2] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge," *Cryptology ePrint Archive*, Paper 2019/953, 2019.

[3] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza, "Succinct non-interactive zero knowledge for a von Neumann architecture," in *Proc. 23rd USENIX Security Symp. (USENIX Security 14)*, pp. 781–796, 2014.

[4] T. Liu, X. Xie, and Y. Zhang, "zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy," *Cryptology ePrint Archive*, Paper 2021/673, 2021.

[5] S. Lee, H. Ko, J. Kim, and H. Oh, "vCNN: Verifiable convolutional neural network based on zk-SNARKs," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 4254–4270, 2024.

[6] Y. Lindell and B. Pinkas, "Secure multiparty computation for privacy-preserving data mining," *JPC*, vol. 1, no. 1, 2009.

[7] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proc. 33rd Int. Conf. Mach. Learn. (ICML)*, pp. 201–210, 2016.

[8] A. Brutzkus, R. Gilad-Bachrach, and O. Elisha, "Low latency privacy preserving inference," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, pp. 812–821, 2019.

[9] M. Mitchell *et al.*, "Model cards for model reporting," in *Proc. Conf. Fairness, Accountability, and Transparency (FAT*'19)*, pp. 220–229, 2019.

[10] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, "Datasheets for datasets," 2021.

[11] F. Dernoncourt, J. Y. Lee, O. Uzuner, and P. Szolovits, "De-identification of patient notes with recurrent neural networks," *J. Amer. Med. Inform. Assoc.*, vol. 24, no. 3, pp. 596–606, 2017.